Frontend screens

React 19 + TypeScript SPA (`frontend/src`), routed with React Router 7 (`App.tsx`). Server state is TanStack Query (`retry: false`, `refetchOnWindowFocus: false`); the auth token lives in `localStorage` (`hooks/useAuth.tsx`). In development the Vite server proxies `/api/*` to `127.0.0.1:8000` and strips the prefix (`vite.config.ts`), so `API_BASE_URL` defaults to `/api` (override with `VITE_API_BASE_URL`). Every page except Login is loaded on demand (`React.lazy` in `App.tsx`), so the first download is the ~360 kB main bundle plus the page being opened.

Platform admins pick which company they are looking at with the **company picker** (`CompanyPicker`, `hooks/useActingCompany.tsx`, remembered per browser) on the case queue, dashboard, audit history and the company settings pages; an amber note reminds them the access is read-only and recorded in the platform audit log.

`ProtectedRoute` redirects to `/login` if there is no token or the token does not resolve to a user. `RoleRoute` (in `components/AdminRoute.tsx`) shows an "access required" panel to the wrong role. **Both are conveniences — the backend independently 401/403s every call**, so a user who reaches a URL still cannot read or change anything they are not entitled to.

Route map

Route	Screen	Access
`/login`	LoginPage	public
`/`, `/cases`, `/review-queue`	CaseQueuePage	any logged-in user (platform admin: per company, read-only)
`/cases/new`	NewCasePage	company users (hidden for platform admins)
`/cases/bulk`	BulkUploadPage	company users (platform admins see it read-only, without the upload form)
`/bulk-uploads/:bulkUploadId`	BulkUploadDetailPage	the uploader; reviewers of the company; platform admins (per company, read-only)
`/cases/:caseId`	CaseDetailPage	any (reduced view for `user`; read-only decision panel for a Reviewer L1 on an L2 case; "Support view" for a platform admin)
`/dashboard`	DashboardPage	company users
`/my-cases`	MyCasesPage	company users
`/audit-history`	AuditHistoryPage	Reviewer L1 and above; platform admins
`/settings` → `/settings/issuer-registry`	SettingsIssuerRegistryPage	Reviewer L2; platform admins (per company)
`/settings/risk-rules`	SettingsRiskRulesPage	Reviewer L2; platform admins (per company)
`/settings/users`	SettingsUsersPage	platform admins only
`/platform` → `/platform/companies`	PlatformCompaniesPage	platform admins only
`/platform/rule-templates`	PlatformRuleTemplatesPage	platform admins only

Route	Screen	Access
`/platform/usage`	PlatformUsagePage	platform admins only
`/platform/queues`	PlatformQueuesPage	platform admins only
`*`	redirect to `/`	—

The top `Nav` (`design-system/Nav.tsx`) shows each item only to who may use it (`NAV_VISIBLE`): *Dashboard* and *My cases* to company users, *Audit history* to Reviewer L1+ and platform admins, *Settings* to Reviewer L2 and platform admins, *Platform* to platform admins. It shows the company name (or "Platform") next to the user, and hides *New upload* from platform admins. Route guards (`RoleRoute minRole=… platformAdmin="allow" | "only"`) mirror this.

LoginPage — `/login`

Purpose: email/password sign-in. **Endpoints:** `POST /auth/login` , then `GET /auth/me` (via `useAuth`). **Notes:** stores the JWT in `localStorage` . Deliberately has no "trust this device", SSO button or compliance badges.

CaseQueuePage — `/` , `/cases` , `/review-queue`

Purpose: the case list / reviewer queue. Shows counts (open, awaiting review, cleared, escalated), a table with case number, type, status, flag (risk tier), escalation badge, document count and submitter. Reviewer L1 gets tabs *My queue* (`can_act`) / *Escalated to L2 · view only*; Reviewer L2 gets *All cases* / *Escalated queue (L2)*; rows the user can't act on show a "View only" pill. **Endpoints:** `GET /cases` (once; refetched by other screens via the `["cases", token]` query key). **Behaviour:** search box + status / case-type / flag filters applied **in the browser**. The status column is a UI mapping of `CaseStatus` → `Analyzing` (`submitted` , `under_automated_review`), `Awaiting Review` (`pending_manual_review` and any other open status), `Approved` (`approved` , `auto_approved` , `closed`), `Rejected` . **"Escalated" is a tier filter (L2), not a status.** For Reviewer L2 the API sorts open L2 cases first, then newest first. **Roles:** every company user, but a `user` only receives their own cases and no risk tier (the flag reads "In review"). A platform admin sees one company at a time, chosen with the company picker — never a mixed list.

NewCasePage — `/cases/new`

Purpose: submit a case: choose a case type, attach PDFs, upload. **Endpoints:** `POST /cases` , `POST /cases/{id}/documents` (once per file, XHR with progress), and, for the optional signature step, `GET …/documents/{id}/file-url` and `POST …/documents/{id}/signature-references` (through `SignatureReferenceCreator`). **Flow:** React Hook Form + Zod validate the case type. The drop zone accepts **PDF only, within the company's per-file limit, not password-protected** (it shows the limit, from `GET /auth/me/upload-limits`); a file that is not a PDF, is empty or is over that limit is refused in the browser before anything is sent (`lib/uploadLimits.ts`). The case is created once (so a retry after a failed upload reuses it); files upload in parallel; each shows its own progress, or the server's specific rejection message (e.g. "This file is password-protected. Please remove the password and re-upload."). After upload the user can open **Set reference signature** for any uploaded document (drawing a box over a `react-pdf` page) — optional; without it no comparison runs. Then the case detail page is opened. **Roles:** any company user may create a case, upload to their own case and set reference signatures on it; reviewers may do so on any case of their company. A stranger's request is a 404; platform admins can't upload.

BulkUploadPage — `/cases/bulk`

Purpose: submit many cases at once as one zip with one folder per case (pipeline 01a). **Endpoints:** `POST /bulk-uploads` (the zip as the raw body, XHR with progress), `GET /bulk-uploads` (recent uploads). **Flow:** pick the case type for every case in the zip, then drop one `.zip`. The browser refuses a non-zip, empty or over-limit zip (the company's own zip limit, shown on the page) before sending (`lib/uploadLimits.ts`). It also counts the case folders from the zip's central directory and warns above 100, without blocking. After upload it opens the summary page. Linked from NewCasePage ("Bulk upload (zip of cases)").

BulkUploadDetailPage — `/bulk-uploads/:bulkUploadId`

Purpose: the live per-case summary of one bulk upload. **Endpoint:** `GET /bulk-uploads/{id}`, polled every 3 s until `settled`. **Shows:** zip-level status and any failure, the >100-case warning, the wrapper-folder and ignored-files notes, an overall progress bar with counts per live status, and a table of every case folder: case number (a link once created), documents processed, live status (Checking files / Queued / Processing / Done / Flagged / Not created) and risk flag. Each rejected file is listed under its case with the reason. Filters: All / Needs attention / In progress / Done. Folder and file names render with `dir="auto"` (Arabic).

CaseDetailPage — `/cases/:caseId`

Purpose: everything about one case in three columns (`lg:grid-cols-12`): **documents** (3), the **selected document + its checks** (5), and **risk + decision** (4), plus the activity timeline below. **Endpoints:** `GET /cases/{id}` (polls every **5 s** while `pipeline.complete` is false or there is no assessment), `GET /cases/{id}/audit-log`, `GET /cases/{id}/signature-matches` (polled every 10 s), and, through child components, the decision endpoints and report endpoints. **Contents:**

- *Left:* the case flag card and the document list (type and processing status).
- *Middle:* `PdfOverlayViewer` showing the selected PDF with live overlays from that document's checks and cross-document findings; the document's **core** extracted fields (issuer — the only one with an "(uncertain)" marker — reference number, date, amounts; `additional_fields` are not listed here); `DocumentChecksPanel` (one expandable card per check, plus a cross-document card and the signature comparison card).
- *Right:* score/tier and the explainable findings (`SeverityFinding` per triggered reason, from `assessment.triggered_reasons`), `CaseDecisionPanel`, and `CaseReportExport` (reviewers only).
- *Bottom:* `ActivityTimeline` (the case's audit trail, newest first). **Roles:** everyone can open a case they may see (a foreign case is a 404 → "Case not found"). A `user` receives no assessment, no tier and only approve/reject actions; decision and report controls are reviewer only. A platform admin gets an amber **"Support view — read-only"** banner and no decision or report controls. The header shows an "Escalated · L2" badge on an L2 case; when the API returns `can_act: false` (a Reviewer L1 on an L2 case) the decision panel is a read-only card saying the case is with Reviewer L2, while the report export stays available.

DashboardPage — `/dashboard`

Purpose: at-a-glance operational summary. **Endpoints:** `GET /cases` only. **Everything is computed in the browser** from that list: KPI cards (counts by flag: high/medium/low/pending, escalated open cases, cleared), a risk-distribution panel, the top-5 open cases the user can act on (`can_act` ; escalated first, then highest risk score, then newest), and a "Case Summary" strip (total cases, documents analyzed, escalated, cleared). **Notes:** this is *not* an MIS/compliance dashboard (no fraud rate, resolution time, volumes over time, export). It contains no charts library.

MyCasesPage — `/my-cases`

Purpose: the signed-in user's own cases, with status filters (Under review / Cleared / Action required) and search. **Endpoints:** `GET /cases`; client filters to cases the user submitted. **Notes:** a submitter sees decision outcomes here but not risk detail.

AuditHistoryPage — `/audit-history`

Purpose: read the company's append-only `audit_log` (platform admins: a chosen company's log, or — with the *Platform-level log* checkbox — the platform events: companies, users, template edits, usage reconciliations, platform access). **Endpoints:** `GET /audit-log` (paged; `event_type`, `q` with a debounce, `limit` / `offset`) and `GET /audit-log/event-types` (for the filter). Event descriptions are rendered by `lib/audit.ts` (`describeAuditEvent`), which turns each `event_type` + `event_data` into a sentence. **Roles:** Reviewer L1 and above, and platform admins (`RoleRoute minRole="reviewer_l1" platformAdmin="allow"`; the API also 403s others). Company users never see platform-access rows.

SettingsIssuerRegistryPage — `/settings/issuer-registry`

Purpose: manage the company's issuer registry (name, Arabic name, tax id, type, active). **Endpoints:** `GET/POST /settings/issuers`, `PATCH /settings/issuers/{id}` (deactivate/reactivate — rows are never deleted). **Roles:** Reviewer L2 (own company); platform admins pick a company. Uses `SettingsShell` (tab bar, company picker) and `Modal`.

SettingsRiskRulesPage — `/settings/risk-rules`

Purpose: view and tune the company's scoring rules and thresholds. **Endpoints:** `GET /settings/risk-rules`, `PATCH /settings/risk-rules/{rule_id}` (weight / severity / active — inserts a new version), `GET …/{rule_id}/history` (per-rule change trail), `GET/PUT /settings/risk-thresholds`, and via `AddRuleModal` `GET /settings/risk-rule-options` + `POST /settings/risk-rules`. **Notes:** edits apply to the company's cases scored from then on; historical assessments keep the versions they used. **Roles:** Reviewer L2 (own company); platform admins pick a company.

SettingsUsersPage — `/settings/users`

Purpose: manage every account on the platform (there is no self-service sign-up and no company-level user management). Lives under the *Platform* tabs (`PlatformShell`). **Endpoints:** `GET/POST /settings/users` (via `AddUserModal`), `PATCH /settings/users/{id}` (role, company, active). The table has a Company column and a company filter; *Add user* asks for the company and role. The role picker lists User / Reviewer L1 / Reviewer L2 / Platform Admin (`ROLE_OPTIONS` in `types/auth.ts`; API values `user` / `reviewer_l1` / `reviewer_l2` / `platform_admin`). Moving a user to another company invalidates their current token. A platform admin cannot change their own role or deactivate themselves. **Roles:** platform admins only.

PlatformCompaniesPage — `/platform/companies`

Purpose: list, create, rename, suspend and reactivate companies, and set each company's upload limits. **Endpoints:** `GET/POST /platform/companies`, `PATCH /platform/companies/{id}`. **Notes:** creating a company seeds its risk rules from the platform template and default thresholds; its issuer registry starts empty. Suspending blocks sign-in and existing tokens for all its users. The table shows each company's limits ("10 MB / file · 300 MB / zip"). **Edit** opens a form with the name, **Max file size (MB)** and **Max zip size (MB)**, pre-filled; only changed values are sent, each change is platform-audited (`company_updated`, old → new), and it applies to the company's next upload without a new sign-in.

Form validation (every form): a problem is shown in red directly under the field (`FieldError` + red border), never only as a hover tooltip. Submit buttons stay enabled, so clicking with a problem marks the field instead of silently doing nothing.

PlatformRuleTemplatesPage — `/platform/rule-templates`

Purpose: edit the default rule set that every **new** company starts from. **Endpoints:** `GET/POST /platform/risk-rule-templates` , `PATCH /platform/risk-rule-templates/{rule_id}` (re-uses `AddRuleModal`). **Notes:** template edits never change an existing company's rules.

PlatformUsagePage — `/platform/usage`

Purpose: billing & usage, one row per company: cases created, documents uploaded, files stored, storage added in the period, total storage/files. **Endpoints:** `GET /platform/usage` (this month / last month / last 30 days / this year / all time / custom UTC range), `POST /platform/usage/reconcile` (*Reconcile now*).

PlatformQueuesPage — `/platform/queues`

Purpose: live processing-queue monitor (refreshes every 5 s): waiting / running per queue, oldest waiting task, average / p95 / max wait, run time, consumers online, rate-limiter counters (throttled calls, seconds waited, Azure 429s) and outstanding work per company (the fair-share counters). **Endpoints:** `GET /platform/queues` . See processing-queues.md.